

MILESTONE 1: Model Selection and Architecture – LegalEase

In this milestone, we focus on selecting the optimal generative AI model for creating personalized and accurate legal documents using Google's Gemini model via the Google Generative AI SDK. We also define the system architecture and set up the development environment to support document generation, editing, and exporting in various formats (.docx, .pdf, .txt).

Activity 1.1: Research and Select the Appropriate Generative AI Model

1. Understand the Project Requirements

- LegalEase must generate structured legal documents (e.g., contracts, NDAs, agreements) based on user inputs such as:
 - Document type
 - Parties involved
 - Terms and conditions
 - Effective dates

2. Explore Gemini Model Documentation

- Visit [Google Generative AI Documentation](#) to evaluate available Gemini models.
- Review capabilities of gemini-1.5-pro, especially:
 - Long context window (up to 1 million tokens)
 - Structured prompting with multi-part content
 - Fast generation speed and legal-domain compatibility

3. Evaluate Model Performance

- Benchmarks indicate that gemini-1.5-pro is capable of handling complex formatting, reasoning, and structured text generation, making it suitable for legal content generation.

4. Select the Optimal Model

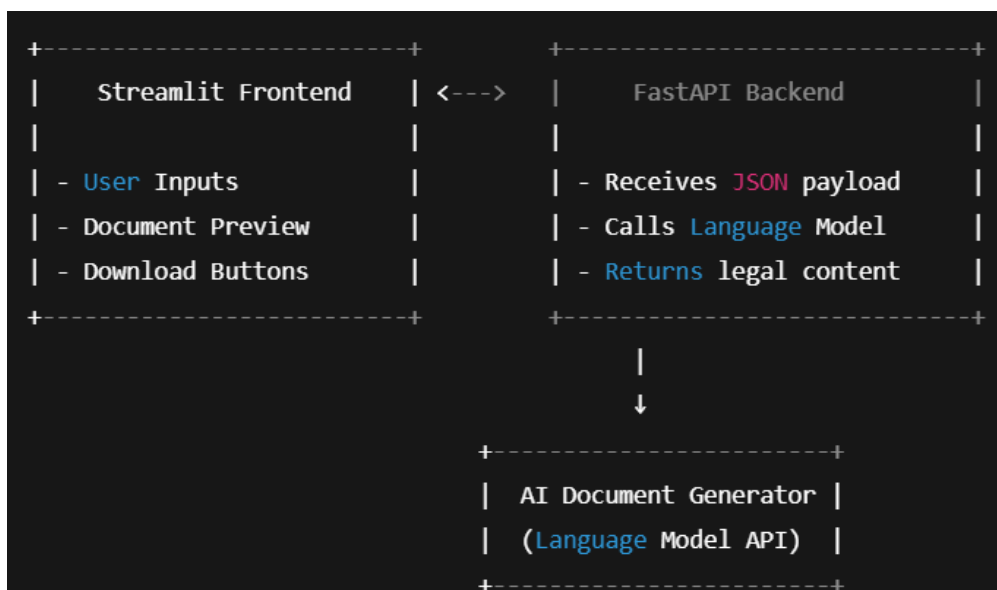
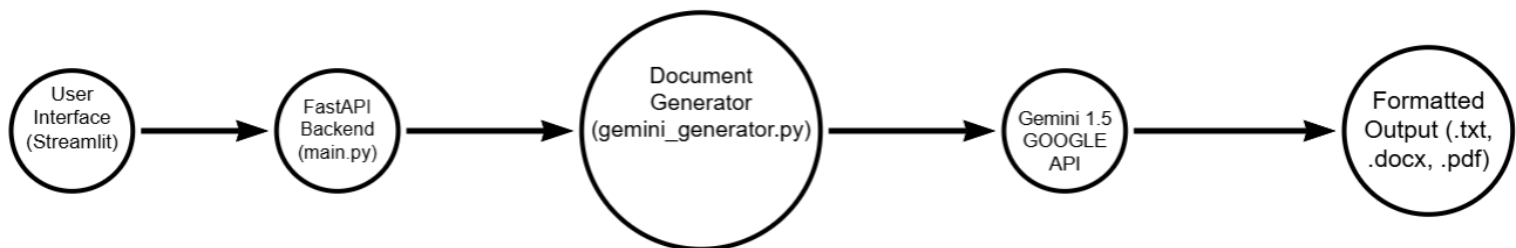
Selected: gemini-1.5-pro

- Best suited for drafting formal documents with clear sections and legal jargon.
- Capable of understanding and generating clauses based on user inputs dynamically.

Activity 1.2: Define the Architecture of the Application

1. Architectural Diagram

- Components:
 - **Frontend:** Streamlit for user interaction (document input, preview, editing, downloads)
 - **Backend:** FastAPI app (routes input to Gemini model and returns generated content)
 - **AI Core:** Gemini integration for content generation
 - **Formatting Modules:** DOCX, PDF, TXT generators



2. Frontend Functionality

- Users provide inputs via:
 - TextInput for document type and dates
 - TextArea for parties and terms
- Buttons:
 - "Generate Document"
 - "Edit Document"
 - "Download as TXT / DOCX / PDF"
- Styled HTML preview of the generated legal document

3. Backend Responsibilities

- Expose POST /generate endpoint (via FastAPI)
- Accept JSON input (document_type, parties, terms, dates)
- Route to GeminiDocumentGenerator
- Return structured document text

4. AI Integration Points

- GeminiDocumentGenerator forms a structured prompt and sends it to Gemini via generate_content
- Receives generated text and returns to FastAPI
- Frontend displays and allows editing or exporting

Activity 1.3: Set Up the Development Environment

1. Install Python and Pip

Ensure Python 3.10+ is installed along with pip for dependency management.

2. Create a Virtual Environment

3. Install FastAPI and Uvicorn (Backend Server)

pip install fastapi uvicorn

4. Install Streamlit (Frontend UI)

```
pip install streamlit
```

5. Install Google Generative AI SDK

```
pip install google-generativeai
```

6. Install Additional Libraries

```
pip install python-docx fpdf Pillow requests python-dotenv
```

7. Set Up Application Structure

